Références lexicales TF-IDF et BM25 sur le corpus travail-emploi

Équipe	Data Tigers
Projet	NLP 2 · Master 2 Data & IA · FGES, Université Catholique de Lille
Ticket	Issue #47 · Branche `feat/47-references-lexicales`
Responsable principal	Remy RAYANE
Soutien et relecture	Mahé BEGNIS
Exécution	4 octobre 2026 (première version) ; 6 octobre 2026 (version relue, dont les résultats suivent)

Table des matières

1 Contexte et objectifs . . . 2
2 Les 5 variantes lexicales étudiées . . . 2
3 Résultats sur les 4 240 passages M2 . . . 3
4 Premiers résultats de BM25 sur des requêtes types . . . 3
5 Arbitrage sur le portage de BM25 en JavaScript (volet A) . . . 4
6 Guide d'intégration dans le notebook 03 (issue #46) . . . 5
7 Historique . . . 5

1 Contexte et objectifs

Le sujet du projet impose de comparer rigoureusement le modèle neuronal de recherche sémantique (`multilingual-e5-small`) aux approches classiques de recherche d'information. Comme le souligne le ticket #47, **BM25 est la seule référence lexicale véritablement exigeante** : une référence mal réglée flatterait l'encodeur.

L'issue #47 a pour objectifs de :

1. construire les moteurs **TF-IDF** (`scikit-learn`) et **BM25** sur les **4 240 passages M2** du découpage structurel du corpus SocialGouv (notebook 01, PR #39) ;
2. étudier **cinq variantes de prétraitement lexical**, en particulier la conservation des négations et des restrictions, essentielles à l'interprétation des règles ;
3. livrer le module `scripts/evaluation/lexical_baselines.py`, prêt pour le notebook 03 (issue #46) ;
4. évaluer le poids d'un portage de **BM25 en JavaScript** dans le site du volet A.

La relecture du 6 octobre a reproduit à l'identique les chiffres de la première version. Elle a corrigé la liste de mots vides, la règle d'ex aequo, la coupure par fréquence de BM25 et la mesure du poids de l'index, et elle a retiré les comparaisons avec l'encodeur qui n'avaient pas été mesurées (section 7). La qualité de recherche de chaque variante, et sa comparaison avec l'encodeur, se mesurent au notebook 03, selon un protocole écrit avant la mesure.

2 Les 5 variantes lexicales étudiées

En droit du travail, le choix de la tokenisation n'est pas neutre :

- les listes usuelles de mots vides retirent des mots comme « ne », « pas », « sans », qui changent le sens d'une règle (« l'employeur ne peut pas licencier sans motif ») ;
- la racinisation regroupe les flexions (*licenciement*, *licenciements*), ce qui réduit le vocabulaire et peut augmenter le rappel.

Variante	Mots vides	Racinisation	Coupure par fréquence	Rôle
V0 (brute)	aucun	aucune	aucune	référence lexicale sans traitement
V1 (Snowball)	liste Snowball (154 mots)	aucune	aucune	référence usuelle
V2 (Snowball sans négations)	liste Snowball, sans « ne », « pas », « sans »	aucune	aucune	conservation du sens restrictif
V3 (racinisation)	comme V2	Snowball française	aucune	regroupement des flexions
V4 (fréquences filtrées)	comme V2	Snowball française	`min_df=2`, `max_df=0.85`	retrait des hapax et des termes omniprésents

- **Liste de mots vides.** C'est celle du projet Snowball, figée sur le même commit et contrôlée par la même empreinte SHA-256 que dans le notebook 01 et la validation sur PIAF (#45). La liste des termes protégés (`TERMES_NEGATION_RESTRICTION`) compte 23 mots, mais seuls « ne », « pas » et « sans » figurent dans la liste Snowball : les 20 autres ne sont retirés par aucune variante.

- **Racinisation.** Algorithme Snowball français, par la bibliothèque `snowballstemmer`, la même que pour PIAF.
- **Coupure par fréquence.** Elle s'applique à BM25 comme à TF-IDF, qui ont toujours exactement le même vocabulaire.
- **Formule de BM25.** Par défaut, celle de Lucene : idf égal à ln(1 + (N - n + 0,5) / (n + 0,5)), toujours positif, avec k1 = 1,2 et b = 0,75. La formule de `rank_bm25`, dont les idf négatifs sont remplacés par 0,25 fois l'idf moyen (k1 = 1,5), reste disponible (`idf="atire"`) : sur les 4 240 passages, elle redonne les scores de `rank_bm25` à 2,7e-12 près. Sur PIAF, la formule de Lucene faisait gagner 1,4 point de Recall@5 (rapport de la PR #55, section 5.3). Le choix de la référence du notebook 03 revient à son protocole.
- **Texte indexé.** L'en-tête du passage (fiche, puis section) et son corps : exactement l'entrée de l'encodeur, sans le préfixe `passage:` .

3 Résultats sur les 4 240 passages M2

Mesures du script `scripts/evaluation/02_references_lexicales.py`. Le poids est celui d'un export JSON réel de l'index BM25 (vocabulaire, idf à 6 décimales, listes inversées, longueurs des passages), puis de sa version gzip. Les temps sont mesurés en Python sur un cœur (Apple M4 Pro), en moyenne sur les 6 requêtes de la section 4 répétées 20 fois : ils servent à comparer les variantes entre elles, pas à prédire le temps dans un navigateur.

Variante	Vocabulaire	Postings	Export JSON	Export gzip	Temps BM25	Temps TF-IDF
V0 (brute)	15 440	384 609	3,55 Mio	0,94 Mio	0,28 ms	0,30 ms
V1 (Snowball)	15 333	290 258	2,76 Mio	0,70 Mio	0,17 ms	0,19 ms
V2 (sans négations)	15 336	293 335	2,78 Mio	0,71 Mio	0,17 ms	0,19 ms
V3 (racinisation)	**8 694**	276 141	2,48 Mio	0,65 Mio	0,29 ms	0,31 ms
V4 (fréquences filtrées)	5 743	273 190	**2,40 Mio**	**0,63 Mio**	0,29 ms	0,31 ms

Ce que ces chiffres montrent :

1. **La racinisation (V3)** réduit le vocabulaire de 43,7 % (de 15 440 à 8 694 racines) et le nombre de postings de 28 % par rapport à V0.
2. **Conserver les négations (V2 contre V1)** coûte 3 mots de vocabulaire et 3 077 postings, soit 0,01 Mio en gzip. Son effet sur le classement n'est pas visible sur les requêtes types (section 4) ; il se mesurera sur le jeu de questions.
3. **La coupure par fréquence (V4)** retire un tiers du vocabulaire de V3, mais seulement 1,1 % des postings : l'export gzip ne gagne que 0,02 Mio.

4 Premiers résultats de BM25 sur des requêtes types

Ces six requêtes, dont les trois premières viennent du notebook 02, illustrent le comportement de BM25 (formule de Lucene). Elles ne mesurent pas sa qualité, et ne disent rien de l'encodeur, qui n'a pas été interrogé ici.

Requête	Premier passage, V3	Rang du passage qui répond (V0, V1, V2, V3)
« Quelles mentions sont interdites sur le bulletin de paie ? »	*Le bulletin de paie > Chapô*, qui annonce seulement que certaines mentions sont interdites	« Et les mentions interdites ? » : 2, 2, 2, 2
« Mon employeur peut-il me licencier pendant un arrêt maladie ? »	*Les absences liées à la maladie ou à l'accident non professionnel > Peut-il y avoir licenciement pour maladie ?*	1 pour les quatre variantes
« Combien de temps peut durer la période d'essai d'un CDI ? »	*Le contrat à durée indéterminée de chantier ou d'opération > Qu'est-ce qu'un contrat de chantier ou d'opération ?*	« Quelle est la durée de la période d'essai ? » : 30, 24, 24, 16
« Quelles sont les dispositions de l'article L. 1221-19 du code du travail ? »	*La période d'essai > Textes de référence*, qui cite l'article	« Quelle est la durée de la période d'essai ? », qui énonce la règle : 6, 5, 5, 6
« Quel est le délai de rétractation lors d'une rupture conventionnelle ? »	*La rupture conventionnelle [...] > La rupture conventionnelle en vidéo*	« Peut-on se rétracter ? » : 5, 4, 4, 2
« Licenciement pour inaptitude sans reclassement »	*La reconnaissance de l'inaptitude médicale [...] > Que se passe-t-il si le reclassement est impossible ou refusé par le salarié ?*	1 pour les quatre variantes

- **Sigles.** La requête sur le CDI échoue : le passage qui répond écrit « contrat de travail à durée indéterminée », jamais « CDI », et BM25 ne rapproche que des mots identiques. La racinisation le remonte du rang 30 au rang 16. Sur cette même question, reprise du notebook 02, l'encodeur plaçait ce passage au rang 5 ; ce n'est qu'une question, et la comparaison se fera au notebook 03 sur tout le jeu.
- **Négations.** Sur « Licenciement pour inaptitude sans reclassement », les trois premiers résultats sont les mêmes pour les cinq variantes : retirer ou garder « sans » ne change rien ici.
- **Références d'articles.** BM25 retrouve en tête un passage qui cite l'article L. 1221-19, mais le passage qui en énonce le contenu n'arrive qu'au rang 5 ou 6.

5 Arbitrage sur le portage de BM25 en JavaScript (volet A)

Le ticket #47 demande s'il faut embarquer un moteur BM25 dans le site statique du volet A :

- **Poids mesuré de l'index (V3) :** 2,48 Mio en JSON, 0,65 Mio en gzip, dans le format minimal décrit en section 3.
- **À comparer avec l'encodeur :** le modèle `multilingual-e5-small` quantifié en q8 pèse 112,8 Mio au premier chargement.

Intérêts possibles, à vérifier :

- **Disponibilité rapide :** BM25 pourrait répondre pendant que le modèle se télécharge. Le temps de chargement et de réponse dans un navigateur reste à mesurer.
- **Moteur de secours :** utile si WebAssembly échoue ou si la machine est trop limitée.
- **Explicabilité :** chaque score se décompose en contributions de mots, ce qui se montre bien en soutenance.

Recommandation pour le Weekly : proposer au domaine D3, dans le cadre de l'issue #48, d'intégrer BM25 en JavaScript comme chargement rapide et comme solution de secours. La décision lui revient.

6 Guide d'intégration dans le notebook 03 (issue #46)

Le module `scripts/evaluation/lexical_baselines.py` s'importe depuis un notebook du même dossier :

```
from lexical_baselines import BM25Retriever, TfidfRetriever
from metrics import evaluate_retrieval, rank_by_score

# variante et formule fixées par le protocole, avant la mesure
bm25 = BM25Retriever(stopwords_variant="snowball_no_negation", stemming="french")
bm25.fit(textes_passages, doc_ids=passage_ids)

# scores de toutes les questions contre tous les passages, pour l'évaluation
scores = bm25.score_matrix(questions_test)

# classements : passages de score positif seulement, ex aequo dans l'ordre de l'index
predictions = bm25.batch_search(questions_test, k=10)
resultats = evaluate_retrieval(predictions, passages_pertinents, k_values=(1, 3, 5, 10))
```

Règles d'usage :

- **Même texte que l'encodeur :** indexer l'en-tête, un saut de ligne, puis le texte du passage : c'est l'entrée de l'encodeur sans son préfixe.
- **Scores nuls :** `search()` et `batch_search()` ne renvoient jamais un passage de score nul. Si une question a moins de 10 passages de score positif, `evaluate_retrieval` lève une erreur ; c'est voulu, et la règle à appliquer (compter l'absence de résultat comme un échec) s'écrit dans le protocole.
- **Paramètres :** aucune recherche d'hyperparamètres sur le jeu de test, qui fausserait la comparaison avec l'encodeur. La variante, la formule, k1 et b se fixent dans le protocole, par exemple d'après PIAF, qui est indépendant de notre jeu.
- **Contrôles :** `python scripts/evaluation/lexical_baselines.py` exécute les contrôles calculés à la main et le recoupement avec `rank_bm25`.

7 Historique

- **4 octobre 2026, première version (Remy RAYANE) :** module `lexical_baselines.py` (BM25 par `rank_bm25`, TF-IDF par `scikit-learn`), script de profilage, cinq variantes, requêtes types, arbitrage du portage en JavaScript.
- **6 octobre 2026, relecture (Mahé BEGNIS) :** chiffres de la première version reproduits à l'identique, puis corrigés comme suit.

Corrections de la relecture :

- liste de mots vides : celle de NLTK (157 mots), présentée comme la liste Snowball, remplacée par la liste Snowball figée (154 mots, 141 en commun) ; la liste NLTK ne contient pas « sans », si bien que l'ancienne V2 ne différait de V1 que par « ne » et « pas » ;
- BM25 réimplanté en matrices creuses : formule de Lucene par défaut, formule de `rank_bm25` recoupée à 1e-9 près, coupure par fréquence appliquée à BM25 (l'ancienne V4 était identique à V3 pour BM25) ;
- règle d'ex aequo commune (`rank_by_score`), et aucun passage de score nul renvoyé, au lieu des premiers passages de l'index pour une requête sans mot connu ;

- poids de l'index mesuré sur un export réel, au lieu d'une estimation présentée comme une mesure ;
- comparaisons avec l'encodeur retirées : sur le bulletin de paie, la section des mentions interdites est au rang 2 pour BM25 et au rang 1 pour l'encodeur ; sur l'article L. 1221-19, l'encodeur place lui aussi en tête un passage qui cite l'article ;
- entrée contrôlée par l'empreinte du manifeste du notebook 01, chemins ancrés à la racine du dépôt, dépendances figées dans `scripts/evaluation/requirements.txt`, contrôles unitaires ajoutés.